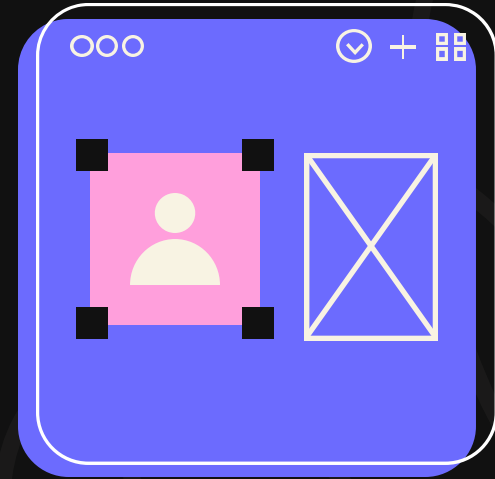


Final Project Report

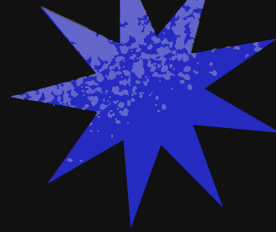
Courtesy of Kelompok 2

LMS



Anggota :

2602050715 - JODY CHRISTIAN
2602054013 - NELSON THEO WILBERT
2602057040 - TOBIAS BENAYA
2602058075 - JOVAN ADELARD WIDODO
2602060426 - VANNESS JACKSON
2602061845 - ERICA NOVITA CHANDRA
2602066833 - WARREN PRESTON HARLIE
2602068656 - FELICIA CAHYA SUSANTO
2602069993 - CLAIRINE
2602072256 - BRANDON AXEL ANGKASA
2602074671 - GALEN NAYAKA NAYOTTAMA
2602081802 - RAFAEL OSVALDO WIJAYA
2602083846 - MARVELLA PANGESTU
2602216310 - EDMUNDUS WISNU PRASETYAJI



What is the Problem ?

</>

7.976 Buku Perpustakaan Malang Belum Kembali, Denda Malah Turun

Zainul Arifin
Diperbarui 06 Okt 2016, 13:15 WIB



Mengatur Inventaris Perpustakaan adalah hal yang sangat Kompleks

Tak Dikembalikan, 2000 Buku di Perpustakaan Jombang Hilang



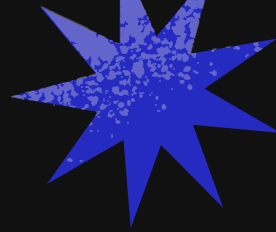
Ilustrasi dalam Perpustakaan Malang, Jombang. (FOTO: ANS)

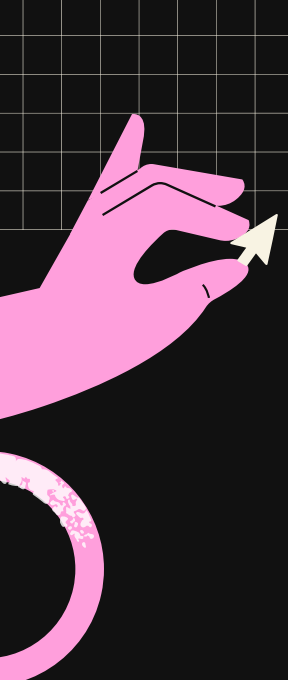
17 Mei 2016, 20:44 WIB
Penulis: Anif Anas
Editor: Anif Anas



Tanpa adanya sistem yang baik, hal ini akan sangat mudah untuk terjadi

Library Management System





CSS

Background and Objectives

Background:

Perpustakaan adalah pusat pengetahuan yang menyediakan akses kepada berbagai bahan bacaan dan sumber ilmu. Namun, dalam era digital ini, tantangan untuk mempertahankan relevansi perpustakaan sebagai lembaga penyedia informasi semakin besar. perpustakaan menghadapi beberapa masalah dan kompleksitas, **diantaranya** :

- manajemen koleksi yang besar
 - peminjaman yang memerlukan efisiensi tinggi
 - kebutuhan akses informasi yang cepat dan akurat bagi pengguna.
- 

Library Management System

Sebuah Sistem Manajemen Perpustakaan , perangkat lunak atau aplikasi yang dirancang untuk membantu dalam pengelolaan dan organisasi berbagai kegiatan di perpustakaan. LMS membantu otomatisasi berbagai tugas dan proses yang terlibat dalam pengelolaan koleksi perpustakaan, peminjaman, pengembalian, dan kegiatan administratif lainnya.

LMS bukan hanya sekadar alat administratif, tetapi sebuah solusi yang mengintegrasikan teknologi untuk meningkatkan efisiensi dan kualitas layanan perpustakaan.



Main Objective



- Meningkatkan efisiensi dalam pengelolaan perpustakaan.
- Memberikan akses cepat dan akurat kepada pengguna
- Meningkatkan pengalaman pengguna dalam pencarian.

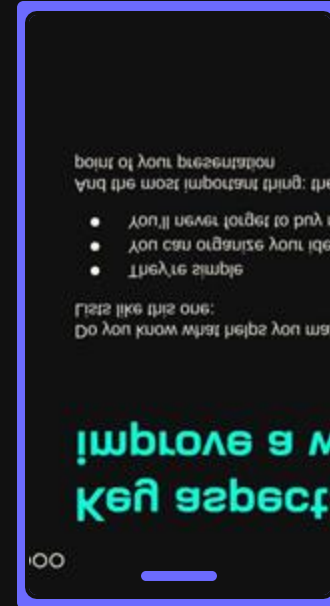


Table of contents

01

Use Case

02

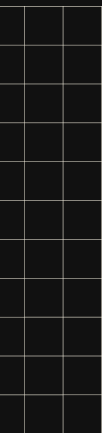
ERD
Activity Diagram

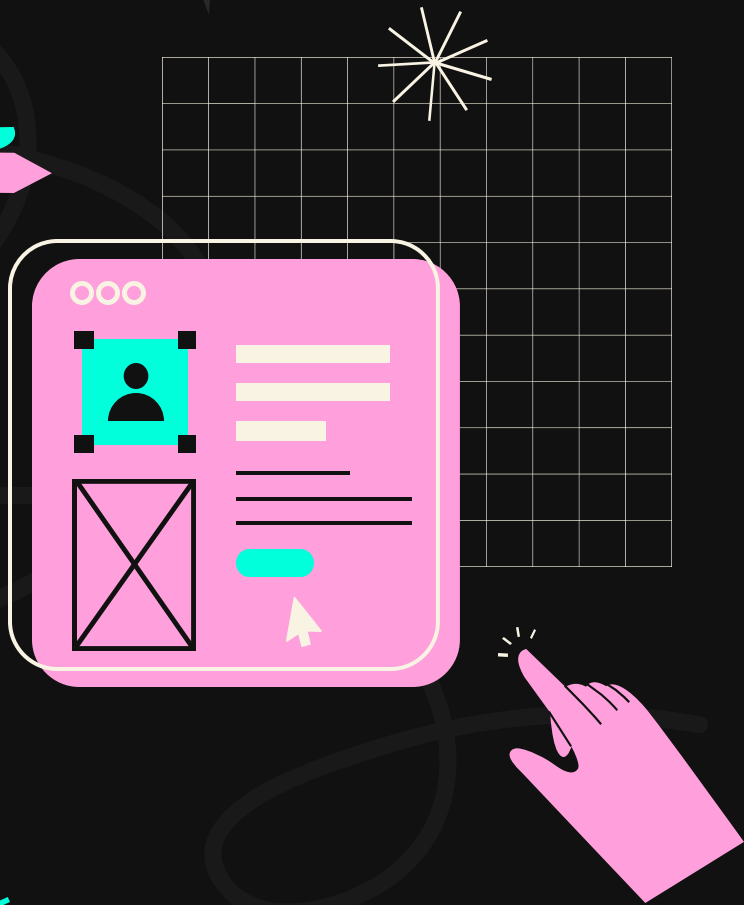
03

Coding Demo

04

Conclusion



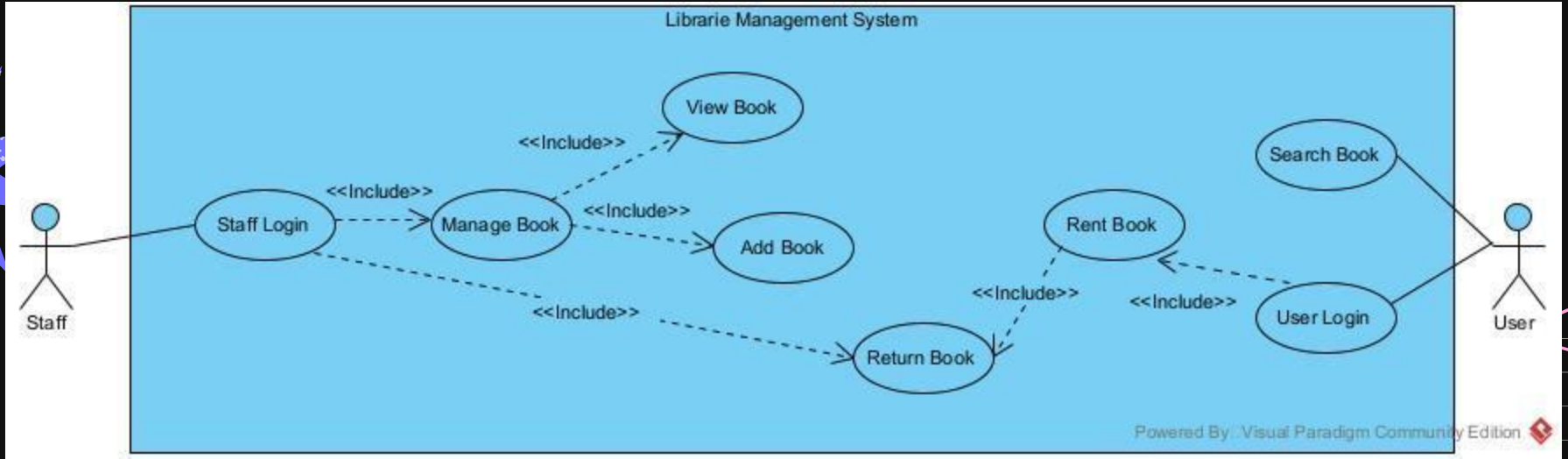


01

Use Case

Use Case

</>

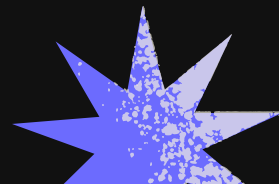




02

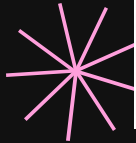
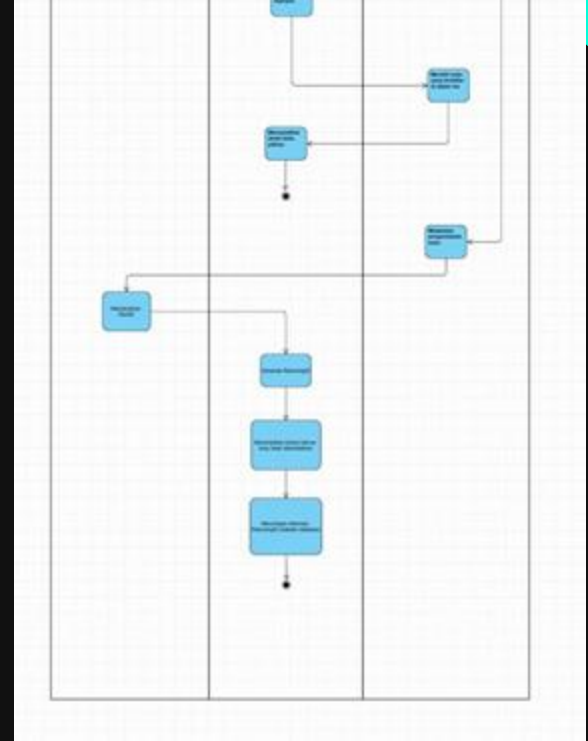
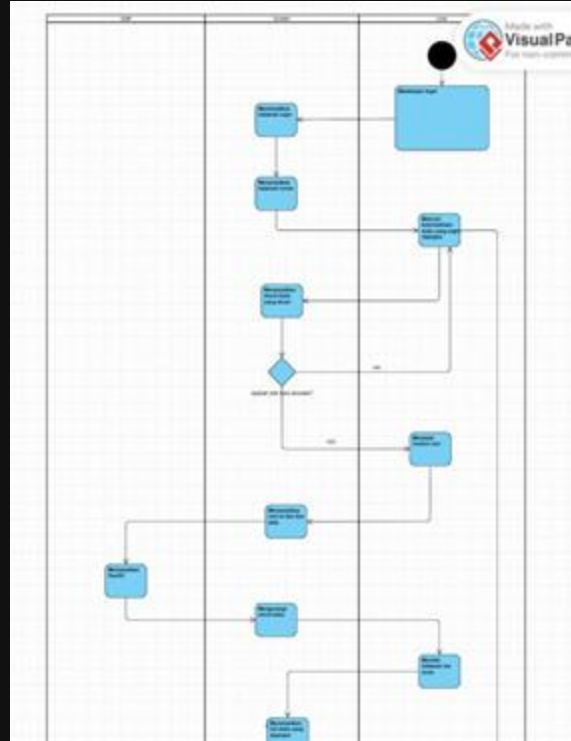
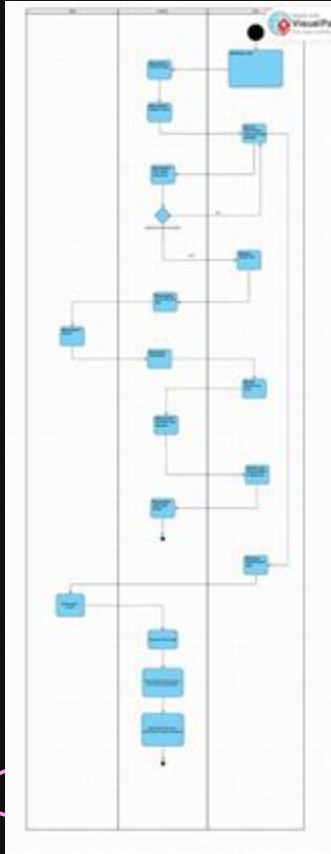
ERD

Activity Diagram

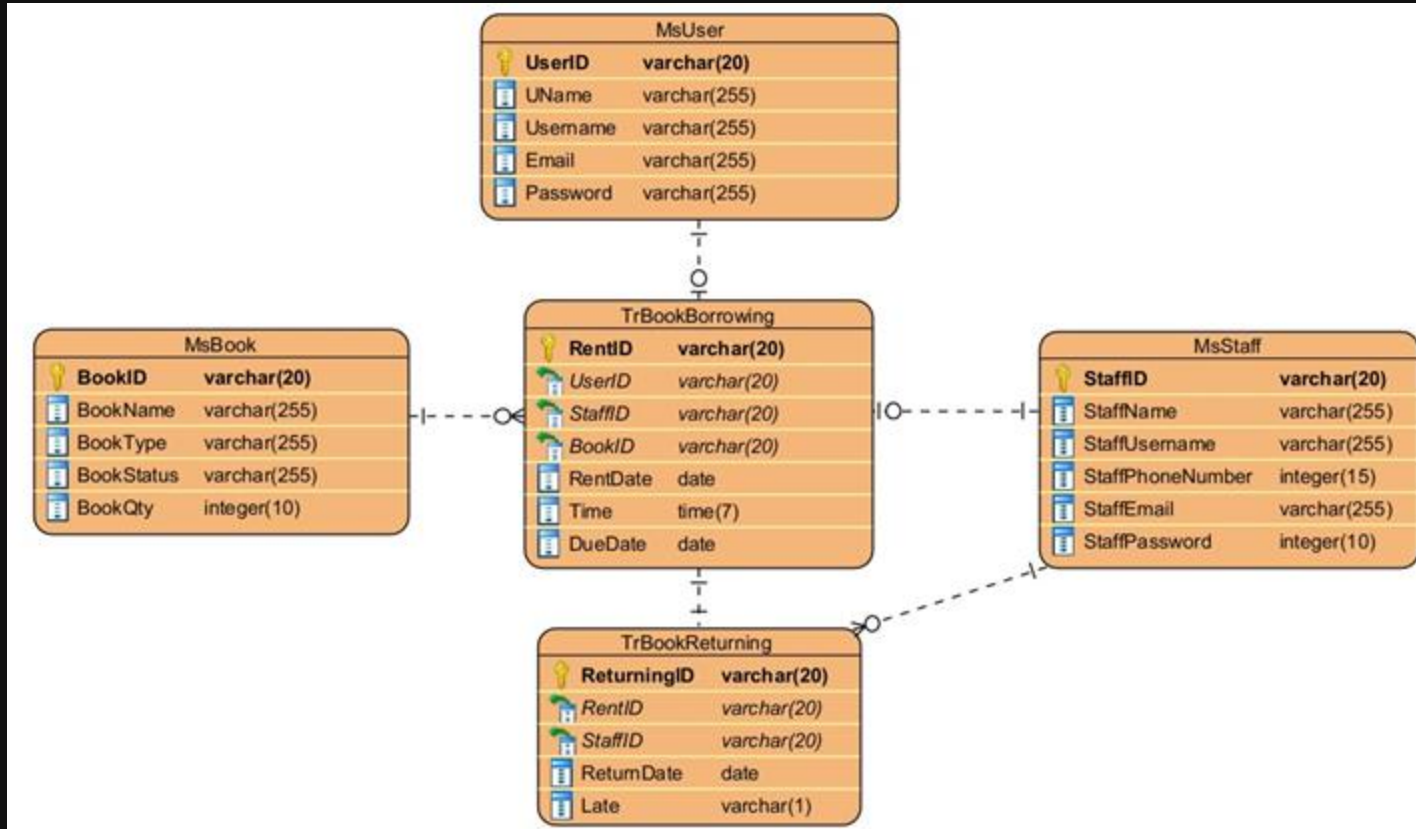


Activity Diagram

</>



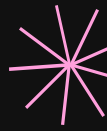
ERD





Sprint Details :

Pre-Development



Kickoff meeting , menentukan use case , menentukan erd relation , membuat activity diagram , dan Sprint coding membuat database dan fitur log in.

Sprint 1

Login feature
Database input
Verifikasi
Penghubungan database

Sprint 2

Fitur input buku dan logout dari sistem aplikasi

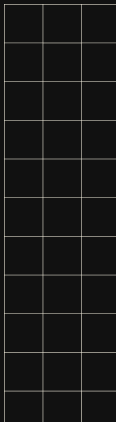


Sprint 3

Fitur penambahan buku
Verifikasi admin yang bisa menambahkan buku
Fitur pencarian buku

Sprint 4

Fitur peminjaman buku oleh customer
Perbaikan tampilan
Perbaikan fitur pencarian buku



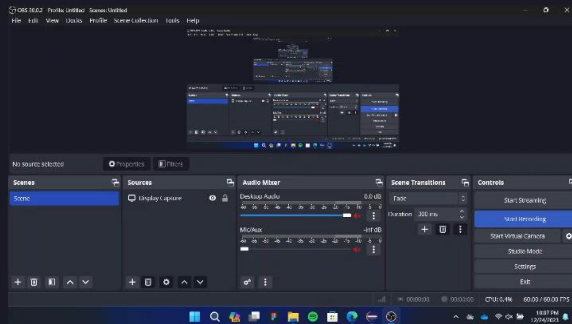
03

Coding Demo



OBS 30.0.2 - Profile: Untitled - Scenes: Untitled

File Edit View Docks Profile Scene Collection Tools Help



No source selected

Properties

Filters

Scenes

Scene

Sources

Display Capture

Audio Mixer

Desktop Audio

0.0 dB

-60 -55 -50 -45 -40 -35 -30 -25 -20 -15 -10 -5 0

Mic/Aux

-inf dB

-60 -55 -50 -45 -40 -35 -30 -25 -20 -15 -10 -5 0

Scene Transitions

Fade

Duration 300 ms

+ -

Controls

Start Streaming

Start Recording

Start Virtual Camera

Studio Mode

Settings

Exit

00:00:00

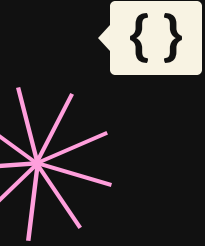
00:00:00

CPU: 0.4%

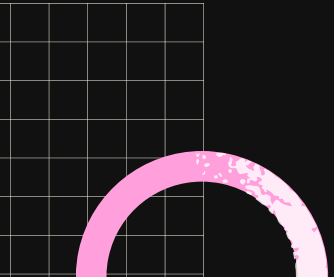
60.00 / 60.00 FPS



10:07 PM
12/24/2023



Coding & Testing



{ }

```
Book.java DatabaseConnection.java LoginScene.java Main.java UserDashboard.java UserService.java module-info.java
1 module YourModuleName {
2     requires javafx.controls;
3     requires javafx.fxml;
4     requires java.sql;
5     requires javafx.graphics;
6     exports main;
7     opens main to javafx.base;
8 }
```

```
Book.java DatabaseConnection.java LoginScene.java Main.java UserDashboard.java UserService.java module-info.java
1 package main;
2
3 import javafx.beans.property.SimpleIntegerProperty;
4
5 public class Book {
6     private final StringProperty judul;
7     private final IntegerProperty stok;
8
9     public Book(String judul, int stok) {
10         this.judul = new SimpleStringProperty(judul);
11         this.stok = new SimpleIntegerProperty(stok);
12     }
13
14     public String getJudul() {
15         return judul.get();
16     }
17
18     public void setJudul(String judul) {
19         this.judul.set(judul);
20     }
21
22     public StringProperty judulProperty() {
23         return judul;
24     }
25
26     public int getStok() {
27         return stok.get();
28     }
29
30     public void setStok(int stok) {
31         this.stok.set(stok);
32     }
33
34     public IntegerProperty stokProperty() {
35         return stok;
36     }
37
38     public void pinjamBuku() {
39         if (stok.get() > 0) {
40             stok.set(stok.get() - 1);
41         }
42     }
43 }
```

</>

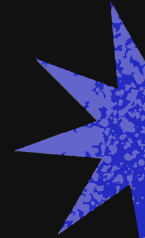
```
Book.java DatabaseConnection.java LoginScene.java Main.java UserDashboard.java UserService.java module-info.java
1 package main;
2
3 import java.sql.Connection;
4
5 public class DatabaseConnection {
6     private static final String DB_URL = "jdbc:mysql://localhost:3306/library";
7     private static final String DB_USER = "root";
8     private static final String DB_PASSWORD = "";
9
10    public Connection getConnection() throws SQLException {
11        return DriverManager.getConnection(DB_URL, DB_USER, DB_PASSWORD);
12    }
13 }
```

Book.java DatabaseConnection.java LoginScene.java Main.java UserDashboard.java UserService.java module-info.java

```
1 package main;
2
3 import javafx.geometry.Insets;
4
5 public class LoginScene extends Scene {
6     private UserService userService;
7     private UserDashboard userDashboard;
8
9     public LoginScene(UserService userService, UserDashboard userDashboard) {
10         super(new HBox(), 800, 400);
11         this.userService = userService;
12         this.userDashboard = userDashboard;
13
14         HBox loginHBox = (HBox) this.getRoot();
15         loginHBox.setSpacing(10);
16         loginHBox.setPadding(new Insets(10));
17
18         Image logoImage = new Image("file:///C:/Users/vanness/Downloads/Compressed/Project-LEC/assets/logo.png");
19         ImageView logoImageView = new ImageView(logoImage);
20         logoImageView.setFitWidth(250); // Per lebar logo
21         logoImageView.setPreserveRatio(true);
22
23         VBox leftVBox = new VBox();
24         leftVBox.setPrefWidth(400);
25         leftVBox.setStyle("-fx-background-color: #FFFFFF; -fx-font-family: 'Poppins'");
26         VBox logoBox = new VBox(logoImageView);
27         logoBox.setAlignment(Pos.CENTER);
28         leftVBox.getChildren().addAll(logoBox);
29
30         VBox rightVBox = new VBox();
31         rightVBox.setPrefWidth(400);
32         rightVBox.setStyle("-fx-font-family: 'Poppins'; -fx-padding: 20;");
33         rightVBox.setAlignment(Pos.CENTER);
34
35         Label titleLabel = new Label("Welcome to Libraris.");
36         titleLabel.setStyle("-fx-font-size: 24px; -fx-text-fill: #000000; -fx-font-weight: bold;");
37
38         GridPane credentialsGridPane = new GridPane();
39         credentialsGridPane.setAlignment(Pos.CENTER);
40         credentialsGridPane.setGap(10);
41         credentialsGridPane.setVgap(10);
42
43         Label usernameLabel = new Label("Username:");
```

```
39 Label usernameLabel = new Label("Username:");
40 usernameLabel.setStyle("-fx-font-family: 'Poppins'");
41 TextField usernameField = new TextField();
42 usernameField.setPrefWidth(150);
43
44 Label passwordLabel = new Label("Password:");
45 passwordLabel.setStyle("-fx-font-family: 'Poppins'");
46 PasswordField passwordField = new PasswordField();
47 passwordField.setPrefWidth(150);
48
49 Button loginButton = new Button("Login");
50 loginButton.setStyle("-fx-background-color: #4F81BD; -fx-text-fill: white; -fx-font-weight: bold; -fx-font-family: 'Poppins'");
51
52 credentialsGridPane.add(usernameLabel, 0, 0);
53 credentialsGridPane.add(usernameField, 0, 1);
54 credentialsGridPane.add(passwordLabel, 1, 0);
55 credentialsGridPane.add(passwordField, 1, 1);
56
57 VBox.setMargin(loginButton, new Insets(10, 0, 0, 0));
58
59 rightVBox.getChildren().addAll(titleLabel, new Separator(), credentialsGridPane, loginButton);
60
61 loginHBox.getChildren().addAll(leftVBox, rightVBox);
62
63 loginButton.setOnAction(e -> {
64     String username = usernameField.getText();
65     String password = passwordField.getText();
66
67     if (userService.validateLogin(username, password)) {
68         userDashboard.setLoggedInUsername(username);
69         String username = userService.getUserByUsername();
70         if ("true".equalsIgnoreCase(username)) {
71             userDashboard.start(Stage) { this.getWindow(); }
72         } else {
73             System.out.println("Nama pengguna tidak valid.");
74         }
75     } else {
76         showErrorAlert("Login gagal", "Periksa username dan password Anda.");
77     }
78 }
79 }
```

</>



Book.java DatabaseConnection.java LoginScene.java Main.java UserDashboard.java x UserService.java module-info.java

```
1 package main;
2
3 import javafx.application.Application;
4
5 public class UserDashboard extends Application {
6
7     private String loggedInUsername;
8     private static final String DB_URL = "jdbc:mysql://localhost:3306/library";
9     private static final String DB_USER = "root";
10    private static final String DB_PASSWORD = "";
11    private TableView<Book> bookTable;
12
13    @Override
14    public void start(Stage primaryStage) {
15        primaryStage.setTitle("User Dashboard");
16
17        VBox dashboardVBox = new VBox();
18        dashboardVBox.setSpacing(10);
19        dashboardVBox.setPadding(new Insets(10));
20        dashboardVBox.setStyle("-fx-background-color: #f0f0f0; -fx-font-family: 'Poppins';");
21
22        HBox menuBarContainer = new HBox();
23        menuBarContainer.setSpacing(10);
24        menuBarContainer.setPadding(new Insets(10));
25
26        Label logoutLabel = new Label("Log Out");
27        logoutLabel.setStyle("-fx-text-fill: white; -fx-font-family: 'Poppins'");
28        logoutLabel.setOnMouseClicked(e -> logout(primaryStage));
29
30        MenuBar menuBar = new MenuBar();
31        menuBar.setStyle("-fx-background-color: #f47061; -fx-text-fill: white; -fx-font-family: 'Poppins'");
32
33        Menu navigationMenu = new Menu();
34        navigationMenu.setStyle("-fx-background-color: #f47061; -fx-font-family: 'Poppins'");
35        navigationMenu.setGraphic(logoutLabel);
36
37        MenuItem logoutMenuItem = new MenuItem();
38        logoutMenuItem.setOnAction(e -> logout(primaryStage));
39
40        navigationMenu.getItems().add(logoutMenuItem);
41    }
42}
```

Book.java DatabaseConnection.java LoginScene.java Main.java x UserDashboard.java UserService.java module-info.java

```
1 package main;
2
3 import javafx.application.Application;
4
5 public class Main extends Application {
6     private UserDashboard userDashboard;
7     private UserService userService;
8
9     public Main() {
10         userDashboard = new UserDashboard();
11         userService = new UserService();
12     }
13
14     public static void main(String[] args) {
15         launch(args);
16     }
17
18     @Override
19     public void start(Stage primaryStage) {
20         primaryStage.setTitle("Library Management System");
21         primaryStage.setScene(new LoginScene(userService, userDashboard));
22         primaryStage.show();
23     }
24 }
25
26
```

</>

Book.java DatabaseConnection.java LoginScene.java Main.java UserDashboard.java x UserService.java module-info.java

```
61 menuBar.getMenus().add(navigationMenu);
62
63 dashboardVBox.getChildren().add(menuBar);
64
65 Region spacer = new Region();
66 HBox.setHgrow(spacer, Priority.ALWAYS);
67
68 menuBarContainer.getChildren().addAll(spacer, menuBar);
69 dashboardVBox.getChildren().add(menuBarContainer);
70
71
72
73 Label welcomeLabel = new Label("Selamat datang, " + loggedInUsername);
74 welcomeLabel.setStyle("-fx-font-size: 20px; -fx-text-fill: #333; -fx-font-family: 'Poppins:');
75
76 TextField searchField = new TextField();
77 Button searchButton = new Button("Cari Buku");
78 HBox searchBox = new HBox(searchField, searchButton);
79 searchBox.setSpacing(10);
80
81 bookTable = new TableView<>();
82 TableColumn<Book, String> titleColumn = new TableColumn<>("Judul Buku");
83 titleColumn.setCellValueFactory(new PropertyValueFactory<>("judul"));
84 bookTable.getColumns().add(titleColumn);
85 TableColumn<Book, Integer> stockColumn = new TableColumn<>("Stok");
86 stockColumn.setCellValueFactory(new PropertyValueFactory<>("stok"));
87 bookTable.getColumns().add(stockColumn);
88
89 HBox buttonBox = new HBox();
90 buttonBox.setSpacing(10);
91 buttonBox.setAlignment(Pos.CENTER);
92 HBox.setHgrow(buttonBox, Priority.ALWAYS);
93
94
95 Button addButton;
96 if (!isStaffUser()) {
97     addButton = new Button("Tambah Buku");
98     addButton.setStyle("-fx-background-color: #4CAF50; -fx-text-
99
100     addButton.setOnAction(e -> {
101         TextInputDialog dialog = new TextInputDialog();
102         dialog.setTitle("Tambah Buku");
103         dialog.setHeaderText("Masukkan detail buku");
```

```
Book.java DatabaseConnection.java LoginScene.java Main.java UserDashboard.java x UserService.java module-info.java
103 dialog.setHeaderText("Masukkan detail buku");
104 dialog.setContentText("Judul Buku");
105
106 TextField judulField = new TextField();
107 TextField stockField = new TextField();
108
109 VBox content = new VBox(0, new Label("Judul Buku:"), judulField, new Label("Stok Buku:"), stockField);
110
111 dialog.getDialogPane().setContent(content);
112 Optional<String> result = dialog.showAndWait();
113 if (result.isPresent()) {
114     String judulBuku = judulField.getText().trim();
115     String stockInput = stockField.getText().trim();
116
117     if (!judulBuku.isEmpty() && !stockInput.isEmpty()) {
118         try {
119             int stok = Integer.parseInt(stockInput);
120
121             if (tambahBukuKeDatabase(judulBuku, stok)) {
122                 showInformationDialog("Sukses", "Buku berhasil ditambahkan ke dalam sistem.");
123                 refreshListView();
124             } else {
125                 showErrorDialog("Kesalahan", "Penambahan buku gagal. Silakan coba lagi.");
126             }
127         } catch (NumberFormatException ex) {
128             showErrorDialog("Kesalahan", "Masukkan stok buku dengan nilai numerik.");
129         }
130     } else {
131         showErrorDialog("Kesalahan", "Judul buku dan stok buku harus diisi.");
132     }
133 }
134
135 } else {
136     addButton = new Button("Pinjam Buku");
137     addButton.setStyle("-fx-background-color: #4CAF50; -fx-text-fill: white; -fx-font-family: 'Poppins:');
138     addButton.setDisable(true);
139     addButton.setOnAction(e -> {
140         Book selectedBook = bookTable.getSelectionModel().getSelectedItem();
141         if (selectedBook != null) {
142             if (hapusBukuDariDatabase(selectedBook)) {
143                 String transactionID = generateTransactionID();
144                 Alert alert = new Alert(Alert.AlertType.INFORMATION);
145                 alert.setTitle("Sukses");
```



```

Book.java DatabaseConnection.java LoginScene.java Main.java UserDashboard.java X UserService.java module-info.java
145 alert.setTitle("Buknes");
146 alert.setHeaderText(null);
147 alert.setContentText("Transaksi " + transactionID + " berhasil. Buku berhasil dipinjam.");
148 alert.showAndWait();
149
150 bookTable.getItems().remove(selectedBook);
151
152 refreshListView();
153 } else {
154     Alert alert = new Alert(Alert.AlertType.ERROR);
155     alert.setTitle("Kesalahan");
156     alert.setHeaderText(null);
157     alert.setContentText("Pinjaman buku gagal. Silakan coba lagi.");
158     alert.showAndWait();
159 }
160 } else {
161     Alert alert = new Alert(Alert.AlertType.WARNING);
162     alert.setTitle("Peringatan");
163     alert.setHeaderText(null);
164     alert.setContentText("Silakan pilih buku yang ingin dipinjam.");
165     alert.showAndWait();
166 }
167 }
168 }
169
170 bookTable.getSelectionModel().selectedItemProperty().addListener((obs, oldSelection,
171     if (newSelection != null) {
172         actionButton.setDisable(false);
173     } else {
174         actionButton.setDisable(true);
175     }
176 });
177
178 HBox.setHgrow(actionButton, Priority.ALWAYS);
179 HBox.setMargin(actionButton, new Insets(0, 10, 0, 0));
180 actionButton.setMaxWidth(Double.MAX_VALUE);
181
182 buttonBox.getChildren().add(actionButton);
183
184 dashboardVBox.getChildren().addAll(welcomeLabel, searchBox, bookTable, buttonBox);
185
186 searchButton.setOnAction(e -> {
187     String searchTerm = searchField.getText();

```

```

Book.java DatabaseConnection.java LoginScene.java Main.java UserDashboard.java X UserService.java module-info.java
187 String searchTerm = searchField.getText();
188     searchBooks(searchTerm);
189 });
190
191 loadBooks();
192
193 Scene dashboardScene = new Scene(dashboardVBox, 800, 400);
194
195 primaryStage.setScene(dashboardScene);
196 primaryStage.show();
197 }
198
199 private void showInformationDialog(String title, String contentText) {
200     Alert alert = new Alert(Alert.AlertType.INFORMATION);
201     alert.setTitle(title);
202     alert.setHeaderText(null);
203     alert.setContentText(contentText);
204     alert.showAndWait();
205 }
206
207 private void showErrorDialog(String title, String contentText) {
208     Alert alert = new Alert(Alert.AlertType.ERROR);
209     alert.setTitle(title);
210     alert.setHeaderText(null);
211     alert.setContentText(contentText);
212     alert.showAndWait();
213 }
214
215 private String generateTransactionID() {
216     int randomDigits = (int) (Math.random() * 1000);
217     return String.format("TRX%03d", randomDigits);
218 }
219
220 private boolean pinjamBuku(Book selectedBook) {
221     try {
222         Connection connection = DriverManager.getConnection(DB_URL, DB_USER, DB_PASSWORD);
223
224         if (selectedBook.getStok() > 0) {
225             String query = "UPDATE buku SET stok = stok - 1 WHERE judul = ?";
226             try (PreparedStatement preparedStatement = connection.prepareStatement(query)) {
227                 preparedStatement.setString(1, selectedBook.getJudul());
228                 int rowsAffected = preparedStatement.executeUpdate();
229

```

```

Book.java DatabaseConnection.java LoginScene.java Main.java UserDashboard.java x UserService.java module-info.java
225
226     preparedStatement.close();
227     connection.close();
228
229     return rowsAffected > 0;
230 }
231 } else {
232     connection.close();
233     return false;
234 }
235 } catch (SQLException e) {
236     e.printStackTrace();
237     return false;
238 }
239 }
240
241 private boolean hapusBukuDariDatabase(Book selectedBook) {
242     try {
243         Connection connection = DriverManager.getConnection(DB_URL, DB_
244
245         if (selectedBook.getStok() > 0) {
246             String query = "UPDATE buku SET stok = stok - 1 WHERE judul
247             try (PreparedStatement preparedStatement = connection.prepa
248             preparedStatement.setString(1, selectedBook.getJudul());
249             int rowsAffected = preparedStatement.executeUpdate();
250
251             preparedStatement.close();
252             connection.close();
253
254             return rowsAffected > 0;
255         } else {
256             connection.close();
257             return false;
258         }
259     } catch (SQLException e) {
260         e.printStackTrace();
261         return false;
262     }
263 }
264
265
266
267
268
269
270
271

```

```

Book.java DatabaseConnection.java LoginScene.java Main.java UserDashboard.java x UserService.java module-info.java
271
272 private boolean isStaffUser() {
273     if (loggedInUsername == null) {
274         return false;
275     }
276     try {
277         Connection connection = DriverManager.getConnection(DB_URL, DB_USER, DB_PASSWORD);
278
279         String query = "SELECT role FROM user WHERE username = ?";
280         try (PreparedStatement preparedStatement = connection.prepareStatement(query)) {
281             preparedStatement.setString(1, loggedInUsername);
282
283             try (ResultSet resultSet = preparedStatement.executeQuery()) {
284                 if (resultSet.next()) {
285                     String userRole = resultSet.getString("role");
286                     return "staff".equals(userRole);
287                 }
288             }
289         } catch (SQLException e) {
290             e.printStackTrace();
291         }
292     }
293     return false;
294 }
295
296 private void searchBooks(String searchTerm) {
297     bookTable.getItems().clear();
298     List<Book> searchResults = performSearchInDatabase(searchTerm);
299     bookTable.getItems().addAll(searchResults);
300 }
301
302 private List<Book> performSearchInDatabase(String searchTerm) {
303     List<Book> searchResults = new ArrayList<>();
304     try {
305         Connection connection = DriverManager.getConnection(DB_URL, DB_USER, DB_PASSWORD);
306
307         String query = "SELECT * FROM buku WHERE judul LIKE ?";
308         System.out.println("SQL Query: " + query);
309
310         try (PreparedStatement preparedStatement = connection.prepareStatement(query)) {
311

```

```

Book.java DatabaseConnection.java LoginScene.java Main.java UserDashboard.java x UserService.java module-info.java
310 String query = "SELECT * FROM buku WHERE judul LIKE ?";
311 System.out.println("SQL Query: " + query);
312
313 try (PreparedStatement preparedStatement = connection.prepareStatement(query)) {
314     preparedStatement.setString(1, "%" + searchTerm + "%");
315
316     try (ResultSet resultSet = preparedStatement.executeQuery()) {
317         while (resultSet.next()) {
318             String judul = resultSet.getString("judul");
319             int stok = resultSet.getInt("stok");
320             searchResults.add(new Book(judul, stok));
321         }
322     }
323 }
324 } catch (SQLException e) {
325     e.printStackTrace();
326 }
327
328 return searchResults;
329 }
330
331 public void setLoggedInUsername(String username) {
332     loggedInUsername = username;
333 }
334
335 private void logout(Stage primaryStage) {
336     loggedInUsername = null;
337     Main loginApp = new Main();
338     Stage loginStage = new Stage();
339     loginApp.start(loginStage);
340     primaryStage.close();
341 }
342
343 private void refreshListView() {
344     List<Book> allBooks = getAllBooksFromDatabase();
345     bookTable.getItems().setAll(allBooks);
346 }
347
348 private boolean tambahBukuKeDatabase(String judulBuku, int stok) {
349     if (judulBuku == null || judulBuku.trim().isEmpty()) {
350         System.out.println("Judul buku tidak boleh kosong.");
351         return false;
352     }

```

```

Book.java DatabaseConnection.java LoginScene.java Main.java x UserDashboard.java UserService.java module-info.java
352 }
353
354 try {
355     Connection connection = DriverManager.getConnection(DB_URL, DB_USER, DB_PASSWORD);
356     String query = "INSERT INTO buku (judul, stok) VALUES (?, ?)";
357     try (PreparedStatement preparedStatement = connection.prepareStatement(query)) {
358         preparedStatement.setString(1, judulBuku);
359         preparedStatement.setInt(2, stok);
360
361         int rowsAffected = preparedStatement.executeUpdate();
362
363         if (rowsAffected > 0) {
364             System.out.println("Book added successfully: " + judulBuku + " - " + stok);
365             refreshListView();
366             return true;
367         } else {
368             System.out.println("Failed to add book to the database.");
369             return false;
370         }
371     } catch (SQLException e) {
372         e.printStackTrace();
373         System.err.println("Error adding book to database: " + e.getMessage());
374         return false;
375     }
376 }
377
378 private void loadBooks() {
379     bookTable.getItems().clear();
380     List<Book> allBooks = getAllBooksFromDatabase();
381     bookTable.getItems().addAll(allBooks);
382 }
383
384 private List<Book> getAllBooksFromDatabase() {
385     List<Book> allBooks = new ArrayList<>();
386
387     try {
388         Connection connection = DriverManager.getConnection(DB_URL, DB_USER, DB_PASSWORD);
389
390         String query = "SELECT judul, stok FROM buku";
391         try (PreparedStatement preparedStatement = connection.prepareStatement(query)) {
392             try (ResultSet resultSet = preparedStatement.executeQuery()) {
393                 while (resultSet.next()) {

```


Book.java DatabaseConnection.java LoginScene.java Main.java UserDashboard.java x UserService.java module-info.java

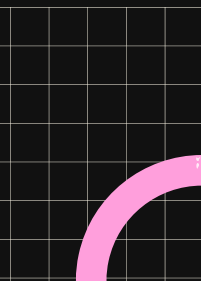
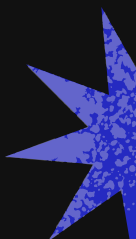
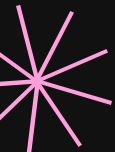
```
365         return true;
366     } else {
367         System.out.println("Failed to add book to the database.");
368         return false;
369     }
370 }
371 } catch (SQLException e) {
372     e.printStackTrace();
373     System.err.println("Error adding book to database: " + e.getMessage());
374     return false;
375 }
376 }
377
378 private void loadBooks() {
379     bookTable.getItems().clear();
380     List<Book> allBooks = getAllBooksFromDatabase();
381     bookTable.getItems().addAll(allBooks);
382 }
383
384 private List<Book> getAllBooksFromDatabase() {
385     List<Book> allBooks = new ArrayList<>();
386
387     try {
388         Connection connection = DriverManager.getConnection(DB_URL, DB_USER,
389
390         String query = "SELECT judul, stok FROM buku";
391         try (PreparedStatement preparedStatement = connection.prepareStatement(
392             try (ResultSet resultSet = preparedStatement.executeQuery()) {
393                 while (resultSet.next()) {
394                     String judul = resultSet.getString("judul");
395                     int stok = resultSet.getInt("stok");
396                     allBooks.add(new Book(judul, stok));
397                 }
398             }
399         } catch (SQLException e) {
400             e.printStackTrace();
401         }
402     }
403
404     return allBooks;
405 }
406 }
407
```

Book.java DatabaseConnection.java LoginScene.java Main.java UserDashboard.java x UserService.java module-info.java

```
1 package main;
2
3 import java.sql.Connection;
4
5 public class UserService {
6     private DatabaseConnection dbConnection;
7
8     public UserService() {
9         dbConnection = new DatabaseConnection();
10     }
11
12     public boolean validateLogin(String username, String password) {
13         try (Connection connection = dbConnection.getConnection()) {
14             String query = "SELECT * FROM user WHERE username = ? AND password = ?";
15             PreparedStatement preparedStatement = connection.prepareStatement(query);
16             preparedStatement.setString(1, username);
17             preparedStatement.setString(2, password);
18
19             ResultSet resultSet = preparedStatement.executeQuery();
20             boolean isValid = resultSet.next();
21
22             resultSet.close();
23             preparedStatement.close();
24
25             return isValid;
26         } catch (SQLException e) {
27             e.printStackTrace();
28             return false;
29         }
30     }
31
32     public String getUserRole(String username) {
33         try (Connection connection = dbConnection.getConnection()) {
34             String query = "SELECT role FROM user WHERE username = ?";
35             PreparedStatement preparedStatement = connection.prepareStatement(query);
36             preparedStatement.setString(1, username);
37
38             ResultSet resultSet = preparedStatement.executeQuery();
39
40             if (resultSet.next()) {
41                 String userRole = resultSet.getString("role");
42                 resultSet.close();
43                 preparedStatement.close();
44             }
45         }
46     }
47 }
```

/>

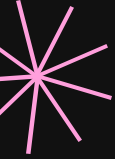




```
Bookjava DatabaseConnection.java LoginScene.java Main.java UserDashboard.java UserService.java x module-info.java
17 String query = "SELECT * FROM user WHERE username = ? AND password = ?";
18 PreparedStatement preparedStatement = connection.prepareStatement(query);
19 preparedStatement.setString(1, username);
20 preparedStatement.setString(2, password);
21
22 ResultSet resultSet = preparedStatement.executeQuery();
23 boolean isValid = resultSet.next();
24
25 resultSet.close();
26 preparedStatement.close();
27
28 return isValid;
29 } catch (SQLException e) {
30     e.printStackTrace();
31     return false;
32 }
33 }
34
35 public String getUserRole(String username) {
36     try (Connection connection = dbConnection.getConnection()) {
37         String query = "SELECT role FROM user WHERE username = ?";
38         PreparedStatement preparedStatement = connection.prepareStatement(query);
39         preparedStatement.setString(1, username);
40
41         ResultSet resultSet = preparedStatement.executeQuery();
42
43         if (resultSet.next()) {
44             String userRole = resultSet.getString("role");
45             resultSet.close();
46             preparedStatement.close();
47             return userRole;
48         }
49
50         resultSet.close();
51         preparedStatement.close();
52
53     } catch (SQLException e) {
54         e.printStackTrace();
55     }
56     return null;
57 }
58 }
59 }
```

</>

Testing



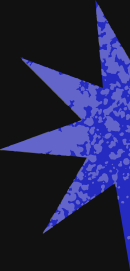
Sprint 1 Testing

Sprint 2 Testing

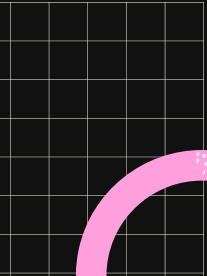


Sprint 3 Testing

Sprint 4 Testing



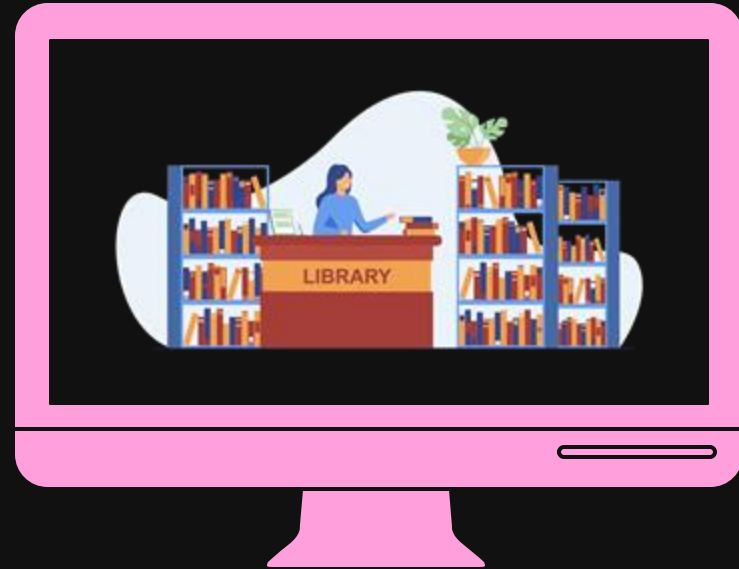
Final
Testing

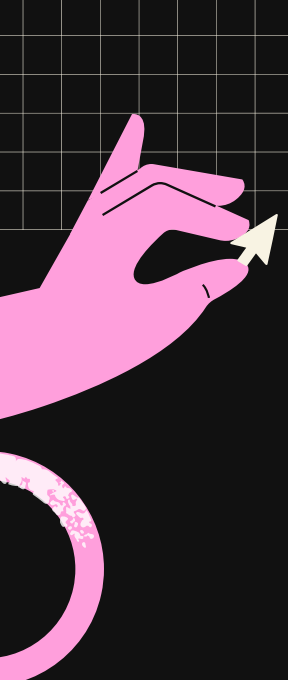


Conclusion

Berikut adalah kesimpulan Project kami :

1. Aplikasi LMS dapat berjalan dengan lancar dengan usecase : login , add book , search book , dan verification.
2. Dalam pengerjaan project tim pasti selalu ada hambatan, namun dengan koordinasi yang tepat hambatan pasti bisa teratasi





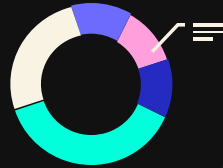
CSS

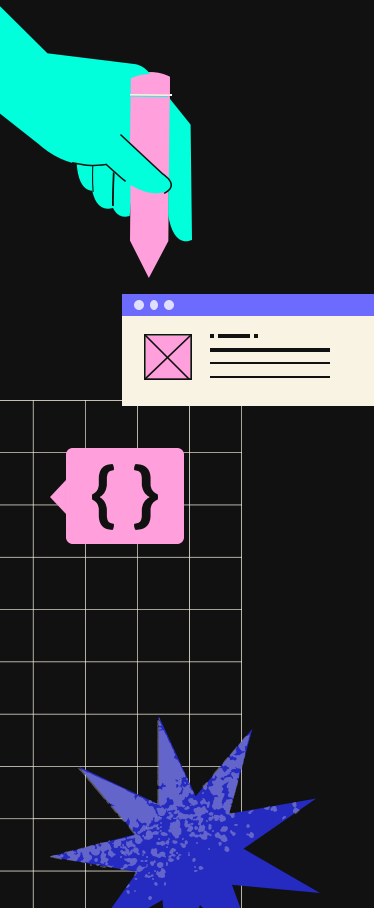
Q&A

- **Apa yang membuat kalian yakin apa yang kalian buat bisa lebih baik dari app library lain?**

Sistem pada aplikasi kami mudah digunakan baik itu oleh pengguna yang memahami teknologi secara luas maupun pengguna baru. Sistem kami juga menyediakan tampilan yang menarik untuk dilihat oleh pengguna.

- **Kenapa admin input data menggunakan judul buku, bukan id buku?**
Karena pada sistem kami id buku tercipta setelah admin memasukkan judul buku terlebih dahulu.





Thanks!

CREDITS: This presentation template was created by [Slidesgo](#), and includes icons by [Flaticon](#), and infographics & images by [Freepik](#)



